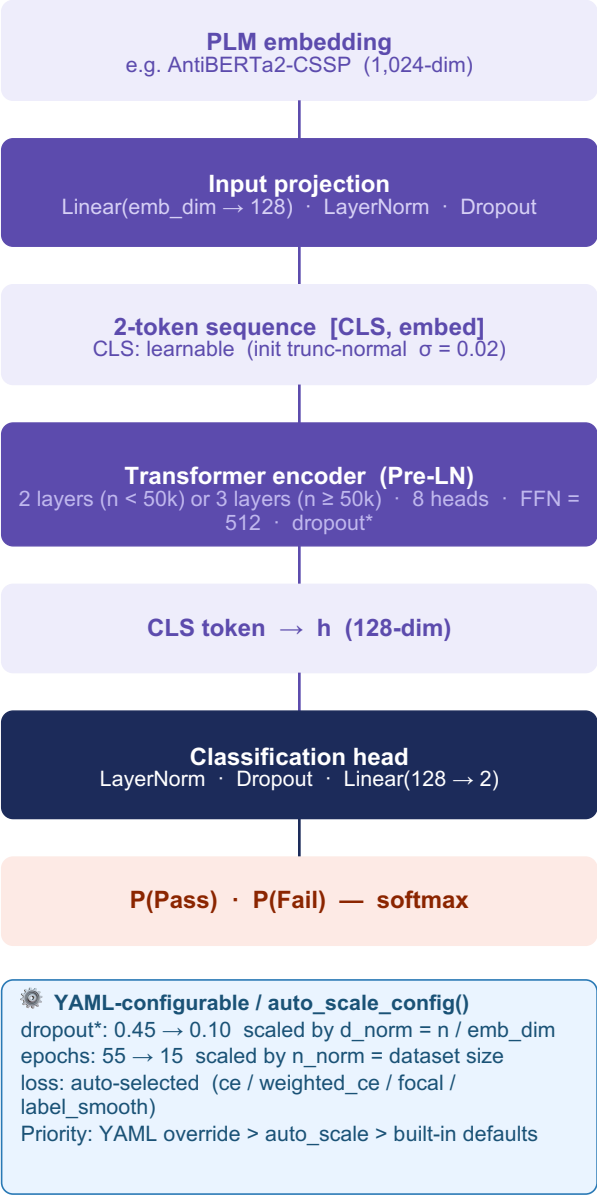


TransformerLM architecture

Fixed-length PLM embedding → 2-token Pre-LN Transformer → P(Pass)



Fixed hyperparameters

hidden_dim	128 (always — PLM head, not a re-learner)
num_heads	8 → head_dim = 16
batch_size	32 (flat minima, better generalisation)
lr	1×10^{-5} (matched to batch = 32)
FFN	512 ($4 \times \text{hidden_dim}$)
Pre-LN	norm_first=True (stable at small batch)

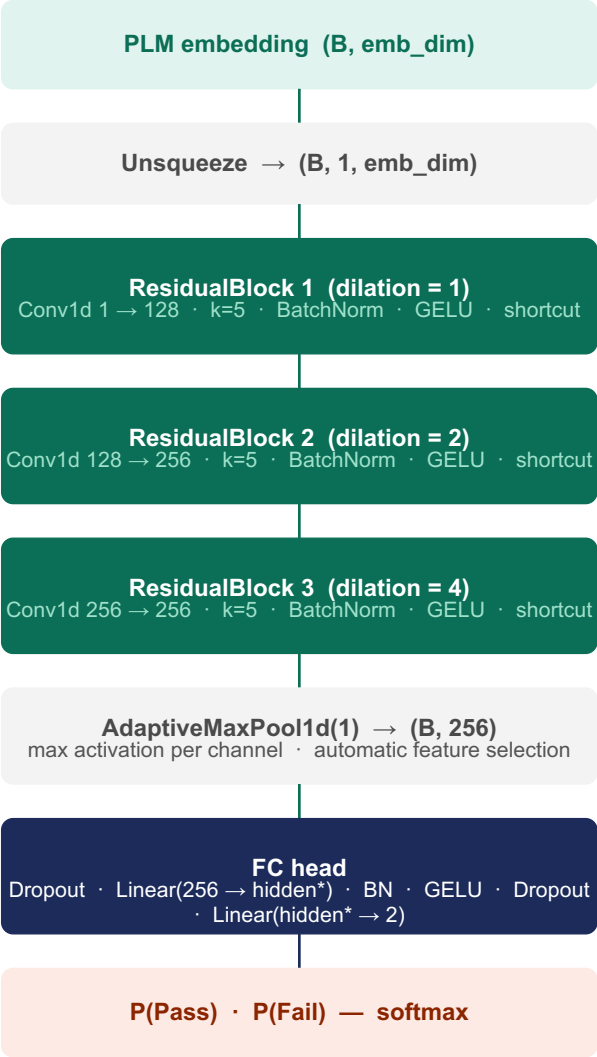
Auto-scaled parameters

dropout*	$0.45 - 0.30 \times d_norm$ ($d_norm = \text{samples}/emb_dim$)
num_layers	2 ($n < 50k$) or 3 ($n \geq 50k$)
epochs	55 → 15 scaled by n_norm
weight_decay	0.001 ($n < 50k$) → 0.0005 ($n \geq 50k$)
patience	7 ($n < 50k$) → 5 ($n \geq 50k$)

Schematic of the TransformerLM classifier. A fixed-length protein language model (PLM) embedding is projected by a Linear(emb_dim → 128) block (LayerNorm + Dropout), then combined with a learnable CLS token (truncated normal initialisation, $\sigma = 0.02$) to form a 2-token input sequence [CLS, embed]. The sequence passes through a Pre-Layer Normalisation (Pre-LN) Transformer encoder producing a CLS token representation that feeds a classification head (LayerNorm → Dropout → Linear(128 → 2)) producing P(Pass) and P(Fail) via softmax. Dashed line: automatic export of the 128-dimensional CLS hidden state as a task-specific embedding CSV for downstream clustering or t-SNE. *Dropout (0.10–0.45) and epoch ceiling (15–55) are scaled automatically by auto_scale_config() using two axes: n_norm (log-scaled dataset size) and d_norm (samples per embedding dimension). All other hyperparameters are fixed across all PLMs and dataset sizes. All parameters are configurable via YAML without code modification (priority: YAML override > auto_scale > built-in defaults).

CNN architecture

Flat PLM embedding treated as 1D signal · dilated residual blocks · AdaptiveMaxPool



Architecture details

kernel_size	5 (all Conv1d layers)
dilation	1, 2, 4 → receptive fields 5, 9, 17
channels	1 → 128 → 256 → 256
shortcut	projection (≠ dims) or identity
pooling	AdaptiveMaxPool1d(1) → 256-dim
BatchNorm	inside every residual block
activation	GELU throughout
†drop_last	True (prevents BN crash at batch_size=1)

Why CNN + TransformerLM ensemble?

CNN extracts multi-scale convolutional features with global max-pooling. TransformerLM uses CLS-guided attention. Both operate on the same PLM embedding but through structurally different computational paths: prediction errors are partially uncorrelated, making them ideal ensemble partners. A simple 0.5 / 0.5 score average is expected to yield AUC ≈ 0.970–0.975.

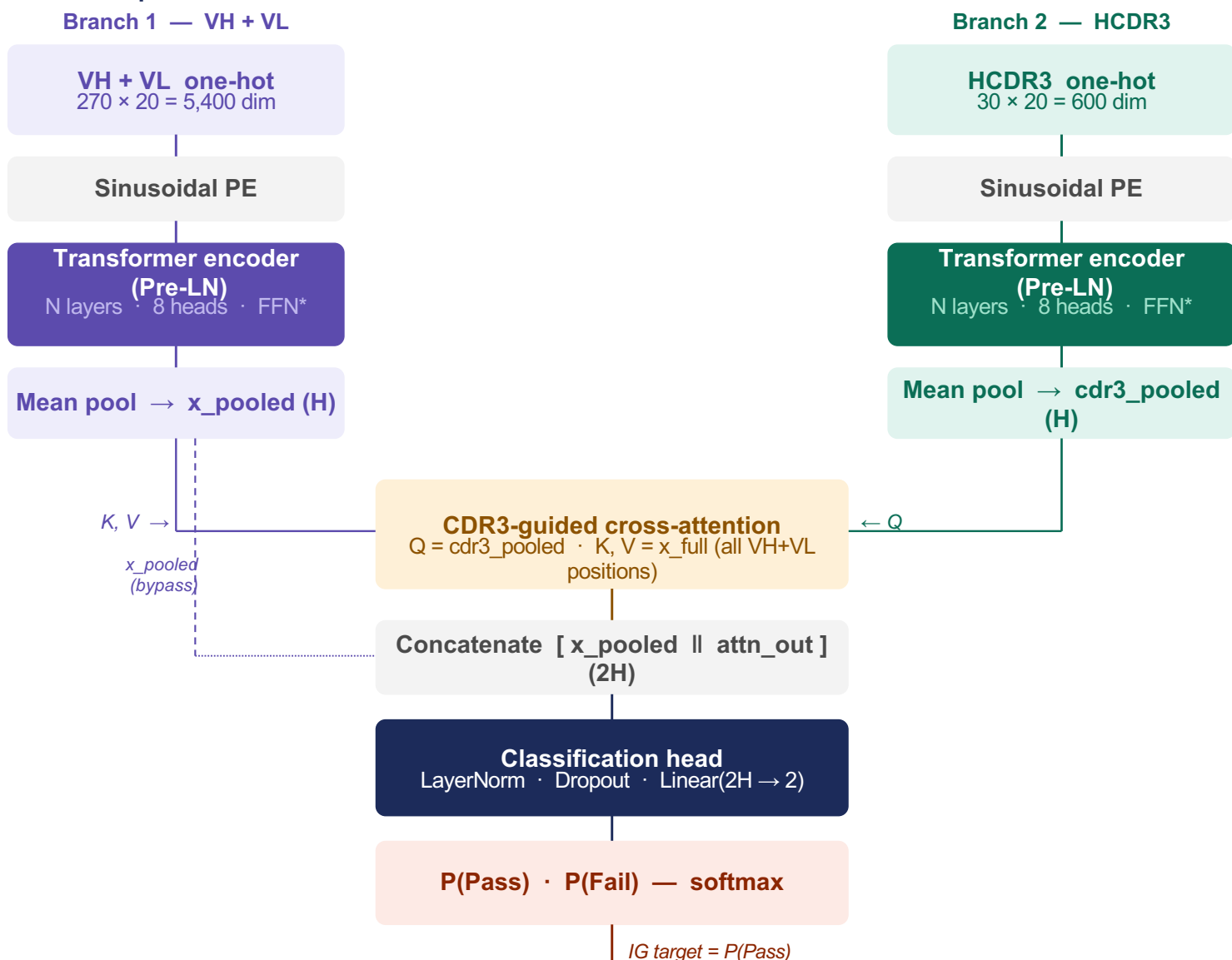
YAML-configurable / auto_scale_config()
hidden_dim*: 64 – 256 scaled by d_norm (n / emb_dim)
dropout: 0.10 – 0.45 scaled by d_norm
batch_size, epochs, lr: scaled by n_norm
loss type: auto (balanced / imbalanced dataset)

Schematic of the 1D residual CNN classifier. A PLM embedding vector ($B \times \text{emb_dim}$) is unsqueezed to $(B, 1, \text{emb_dim})$ and processed by three dilated residual blocks (dilations 1, 2, 4; $\text{kernel_size} = 5$) capturing multi-scale patterns at receptive fields of 5, 9, and 17 positions. Each block applies two Conv1d layers with BatchNorm1d and GELU activation, with a projection shortcut where channel dimensions change. AdaptiveMaxPool1d(1) collapses the feature map to a 256-dimensional vector by selecting the maximum activation per channel — input-conditioned feature selection robust to PLM embedding dimension ordering. The FC head applies Dropout → Linear(256 → hidden*) → BatchNorm1d → GELU → Dropout → Linear(hidden* → 2). †drop_last=True prevents BatchNorm receiving a single-sample batch (undefined variance). Dashed line: penultimate-layer hidden state exported as CSV. *hidden_dim (64–256) and dropout (0.10–0.45) are scaled automatically by auto_scale_config() from d_norm (samples per embedding dimension). All parameters are configurable via YAML without code modification.

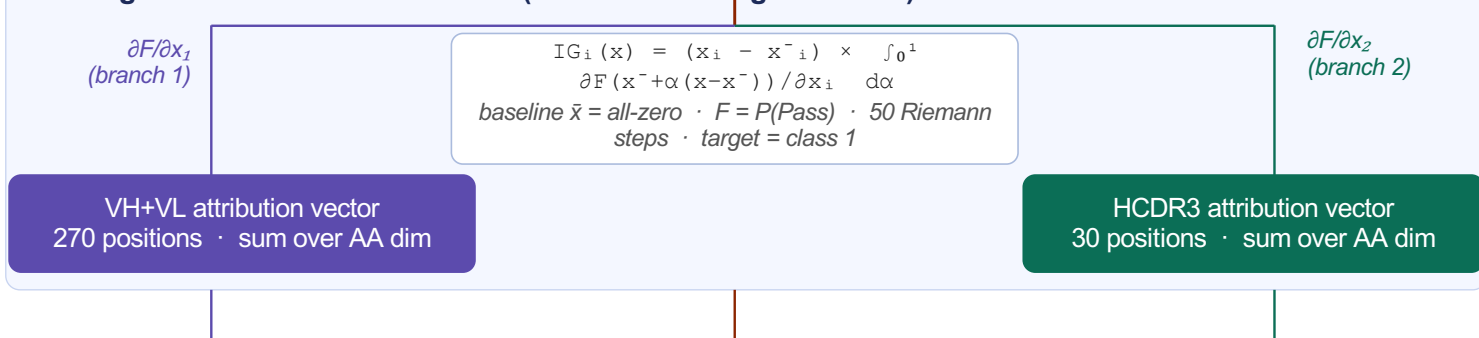
TransformerOneHot: Architecture and Integrated Gradients

Dual-branch one-hot Transformer · CDR3-guided cross-attention · position-resolved IG attribution

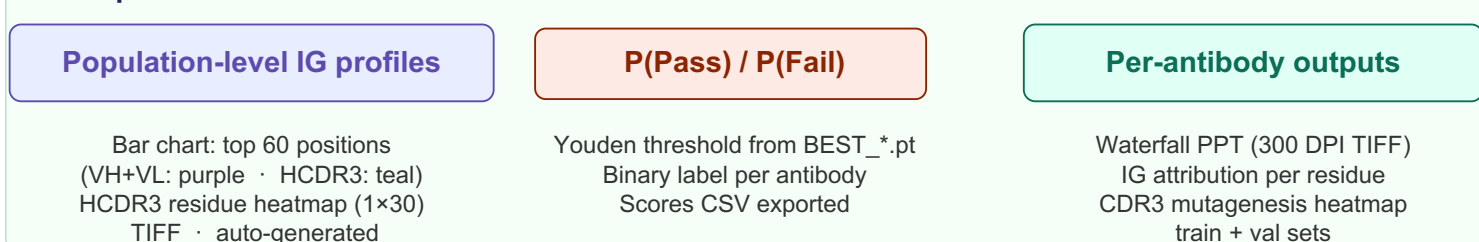
a Forward pass



b Integrated Gradients attribution (backward through network)



c Outputs



YAML-configurable / auto_detect_config(n, pos_rate) N layers · hidden_dim · dropout · FFN*: auto from dataset size + class balance · ig_n_steps (default 50) · top_features: configurable · priority: YAML override > auto_detect > defaults

Transformer_onehot Legend

(a) Forward pass. Branch 1 encodes the concatenated VH + VL sequence as a one-hot position $\times$ amino acid matrix ($270 \times 20 = 5,400$ dimensions). Branch 2 independently encodes HCDR3 ($30 \times 20 = 600$ dimensions). Both branches apply sinusoidal positional encoding and independent Pre-LN Transformer encoder stacks, then mean-pool to x_{pooled} and cdr3_pooled respectively. CDR3-guided cross-attention integrates the branches: cdr3_pooled provides the query (Q) and the full VH+VL sequence provides keys and values (K, V). An x_{pooled} bypass (dotted line) feeds x_{pooled} directly to the concatenation step in parallel with the cross-attention output. The concatenated (2H)-dimensional vector feeds a classification head (LayerNorm $\rightarrow$ Dropout $\rightarrow$ Linear($2H \rightarrow 2$)) producing P(Pass) and P(Fail) via softmax.

(b) Integrated Gradients attribution. Integrated Gradients (IG) computes the path integral of gradients from an all-zero baseline ($\bar{x}$) to the actual input along a straight line, approximated with 50 Riemann steps; attribution target = class 1 (P(Pass)). The coral arrow entering section b at the centre denotes P(Pass) as the IG attribution target. Gradients branch left ($\partial F / \partial x_1$, purple) to Branch 1 and right ($\partial F / \partial x_2$, teal) to Branch 2, propagating backward (upward arrows) through their respective encoder stacks. Summing attributions over the amino acid dimension yields a VH+VL position importance vector (270 IMGT positions, purple box) and an HCDR3 position importance vector (30 IMGT positions, teal box).

(c) Outputs. Classification: P(Pass) scores thresholded at the Youden-optimal threshold embedded in the BEST_*.pt checkpoint after cross-validation. Population-level IG profiles (fed by VH+VL attribution vector): ranked bar chart of the top 60 positions with VH+VL positions coloured purple and HCDR3 positions teal, and a $1 \times \text{CDR3_length}$ HCDR3 residue importance heatmap accumulated as mean absolute IG attributions across all antibodies. Per-antibody outputs (fed by HCDR3 attribution vector): individual IG waterfall plots and CDR3 in silico mutagenesis heatmaps assembled into PowerPoint files (300 DPI TIFF). All outputs generated automatically during --kfold, --train, and --predict without additional user commands.

*N layers, hidden_dim, dropout, and FFN are scaled automatically by auto_detect_config(n, pos_rate) from dataset size and class balance. All parameters are configurable via YAML without code modification (priority: YAML override > auto_detect > built-in defaults). Configurable: loss function, threshold optimisation method, ig_n_steps, top_features, output DPI, and PowerPoint assembly. SGKF, stratified group k-fold; PE, positional encoding; Pre-LN, pre-layer normalisation; IG, Integrated Gradients; PPT, PowerPoint.

Delphi architecture comparison

All models share a consistent public python API · YAML-configurable · auto_scale from data

Model	Input encoding	Key mechanism	Interpretability	Regression
TransformerLM	PLM embedding 480–1,024-dim	2-token Pre-LN Transformer hidden=128 batch=32 lr=1e-5 auto: dropout, epochs	CDR3 mutagenesis hidden state CSV export	No
CNN	PLM embedding 480–1,024-dim	Dilated residual blocks BatchNorm · AdaptiveMaxPool kernel_size=5	CDR3 mutagenesis SHAP Additive exPlanations	No
TransformerOneHot	VH+VL + HCDR3 one-hot (no PLM) 5,400 + 600 dims	CDR3-guided cross-attention dual-branch Pre-LN Transformer auto: N, hidden, dropout	Integrated Gradients position-resolved (270+30) CDR3 mutagenesis	No
Random Forest	k-mer · biophysical one-hot · PLM auto-detected	1,000 trees · OOB FeatureBuilder HCDR3-stratified SGKF	SHAP TreeExplainer waterfall PPT CDR3 mutagenesis	Yes
XGBoost	k-mer · biophysical one-hot · PLM auto-detected	Gradient boosting auto classification / regression HCDR3-stratified SGKF	SHAP TreeExplainer waterfall PPT CDR3 mutagenesis	Yes

⚙️ **All architectures: YAML-configurable without code modification**
auto_scale_config() / auto_detect_config(): hidden_dim, dropout, batch, epochs, lr from n_samples × embedding_dim
Priority: YAML override > auto_scale > built-in defaults | YAML values applied as hard ceilings over auto-scaled values
Configurable: loss (ce/weighted_ce/focal/label_smooth) · threshold (Youden/F2/cost matrix) · SHAP top_n · mutagenesis N · DPI · PPT toggle

Consistent public API: train() · kfold_validation() · predict_proba() · predict() · save() · load()
HCDR3-stratified group cross-validation · Youden threshold embedded in checkpoint · hidden-state CSV export · PowerPoint interpretability output (300 DPI TIFF)

Comparison of the four ML architectures implemented in MLabDev. All architectures share a consistent public API and are configurable via YAML without code modification. auto_scale_config() (neural architectures) and auto_detect_config() (tree architectures) scale hyperparameters from dataset size (n_norm axis) and data density (d_norm = samples per embedding dimension axis). YAML values are applied as hard ceilings over auto-scaled defaults. Random Forest and XGBoost additionally support regression mode for continuous biophysical targets (e.g. PSR normalised score, SEC monomer percentage); task is detected automatically from the label distribution. HCDR3-stratified group cross-validation (Levenshtein 80% identity clusters) is used throughout to prevent sequence leakage. SGKF, stratified group k-fold; PLM, protein language model; SHAP, SHapley Additive exPlanations; IG, Integrated Gradients; PPT, PowerPoint; DPI, dots per inch.